GyanSetu (■■■■■■■■■)

Technical Feature Architecture & Engineering Implementation Guide

System Type: Full-Stack Web Application (SPA)	**Backend:** Spring Boot 3.3.4 (Java 17/21)	**Frontend:** React 19 + TypeScript + Vite
Database: PostgreSQL 14 (Port 5434) + JPA	**Deployment:** Render PaaS (Docker Container)	**Security:** BCrypt + OWASP Defense Headers

1. Executive Architecture Overview

GyanSetu is built as a high-performance, single-container monolithic Single Page Application (SPA). The architecture combines a compiled React 19 frontend served directly through Spring Boot's embedded Tomcat web layer and a robust Java backend connected to PostgreSQL 14. This structure enables a zero-proxy, unified port deployment on Render with sub-second API round-trips and comprehensive client-side routing fallback.

- **Containerized Build:** Multi-stage Docker packaging compiling Vite React assets into src/main/resources/static/ before packaging the executable JAR.
- **State & Resilience Pipeline:** Dual-tier architecture featuring PostgreSQL persistent entities (Users, Profiles, Notifications) and an automated in-memory mock engine allowing seamless offline client demonstration.
- **Defensive Boundary:** Registered servlet filter SecurityHeadersFilter.java enforcing strict CSP, HSTS, frame restriction, and MIME sniffing protection across all public endpoints.

2. Technical Feature Breakdown & Implementation Mechanics

The following sections explain the exact algorithmic and architectural functioning of each technical module in the platform:

Feature 1: Algorithmic AI Eligibility Scoring & Confidence Engine

Components & Source: EligibilityEngineService.java (Backend) | client.ts : evaluateEligibilityMock (Frontend)

Evaluates student profiles against national opportunities using a deterministic multi-criteria rule engine. Unlike simple keyword matchers, the engine calculates a mathematical Confidence Score (0–100%) and classifies candidates into four strict tiers: *ELIGIBLE*, *LIKELY_ELIGIBLE*, *NEEDS_VERIFICATION*, and *NOT_ELIGIBLE*.

Key Mechanics:

- **Age & Reservation Relaxation:** Dynamically adds age relaxation offsets (+3 years for OBC-NCL, +5 years for SC/ST) onto base maximum age limits.
- **Appearing Student Promotion:** Recognizes ongoing enrolled degrees so 3rd/final-year students without completed graduation percentages are qualified for exams like UPSC CSE, GATE, and CAT.
- **Income Ceiling & Quota Enforcement:** Verifies family annual income against statutory thresholds and enforces gender-exclusive schemes (e.g., AICTE Pragati).

Feature 2: Multi-Category Priority Distribution Bar Graphs

Components & Source: PriorityBarGraph.tsx | DashboardOverview.tsx

Renders three independent, symmetrical 4-column bar visualizers across Scholarships, Government Exams, and Higher Studies. The graph sorts top matches in a strictly decreasing slope from left to right, providing an instantaneous visual hierarchy of opportunity relevance.

Key Mechanics:

- **Calibrated Priority Slope:** Computes composite scores factoring match confidence, deadline urgency (extra points for closing within 5–15 days), verification badge, and user bookmarks.
- **A11y & WCAG 2 AA Compliance:** Built with high-contrast palettes (Rosewood, Forest Sage, Slate Navy) achieving >4.5:1 contrast against white backgrounds, accompanied by aria-hidden decorative background guide ticks.
- **Interactive Detail Triggers:** Clicking any bar navigates to the respective filtered category or launches an inspect-eligibility breakdown.

Feature 3: Syllabus Preparation Overlap Detector

Components & Source: OverlapDetectorService.java | PreparationOverlapView.tsx

Solves the primary pain point of competitive exam preparation: redundant effort across multiple examinations. The detector analyzes cross-exam syllabus topics using a tokenized topic intersection matrix.

Key Mechanics:

- **Jaccard Topic Similarity:** Maps overlaps between target exams (e.g., UPSC Prelims CSAT, SSC CGL Tier 1, State PSCs, and RRB NTPC).
- **Common Core Identification:** Highlights shared modules (Quantitative Aptitude, Logical Reasoning, General Awareness) to allow simultaneous preparation.
- **Effort Multiplier Metric:** Quantifies saved study time (e.g., 'Preparing for SSC CGL covers 74% of RRB NTPC syllabus').

Feature 4: Multi-Exam Cost Planner & Exam Date Clash Radar

Components & Source: CostAndClashService.java | CostAndClashView.tsx

Provides financial and logistical foresight for students targeting multiple exams in an academic year. It computes net application fees after statutory fee concessions and detects calendar collisions.

Key Mechanics:

- **Automatic Fee Waiver Deductions:** Applies government fee waivers based on student category (e.g., ■0 for SC/ST or Female applicants in SSC/UPSC), aggregating total savings to date.
- **Total Expense Budgeting:** Estimates travel, lodging, and application expenditures based on examination center distances.
- **Date Collision Warnings:** Flags date overlaps (e.g., GATE vs Banking Mains) so students avoid paying fees for conflicting exams.

Feature 5: What-If Career & Eligibility Simulation Lab

Components & Source: WhatIfSimulatorService.java | WhatIfSimulatorView.tsx

An interactive sandbox allowing students to test hypothetical academic or financial scenarios and immediately view the impact on their career trajectory without modifying their baseline profile.

Key Mechanics:

- **Parameter Perturbation Engine:** Students adjust simulated Class 12th marks, ongoing UG CGPA, expected annual family income, or category certification.
- **Differential Unlocking Pipeline:** Computes the delta of newly unlocked scholarships and competitive exams made accessible by the simulated changes.
- **Target-Oriented Roadmap:** Outlines exact target milestones (e.g., 'Increasing CGPA from 6.2 to 6.8 unlocks 5 additional PSU recruitments').

Feature 6: Document Readiness Matrix & CSC Renewal Tracker

Components & Source: DocumentReadinessService.java | DocumentReadinessView.tsx

Tracks the validity and availability of statutory certificates (Income, Domicile, Non-Creamy Layer, Caste, EWS, Marksheets) necessary for government applications.

Key Mechanics:

- **Lifecycle State Tracking:** Categorizes documents into *VERIFIED*, *EXPIRED*, *IN_PROGRESS*, and *MISSING*.
- **Lead-Time Forecaster:** Factors issuing authority turnaround times (e.g., 14–21 days for Tehsil/CSC income certificates) against upcoming deadlines to prompt proactive renewal.

Feature 7: Official Notice Verification & Scam Radar

Components & Source: TrustAndEngagementService.java | ScamAndTrustView.tsx

Protects vulnerable students from rampant employment and scholarship fraud on social media and unofficial portals.

Key Mechanics:

- **Domain Authenticity Guard:** Validates official URLs against government and statutory domain registries (.gov.in, .nic.in, .ac.in).
- **Scam Pattern Detection:** Flags fake WhatsApp job postings, upfront fee demands for bogus placement drives, and unofficial mirrored portals.

Feature 8: Scheduled Deadline Scanner & Action Alert Polling

Components & Source: DeadlineScannerService.java | DeadlineAlertScheduler.java | NotificationPopover.tsx

An automated monitoring pipeline that ensures no critical closing date or eligibility window is missed.

Key Mechanics:

- **Cron Scheduler:** Executes daily scans (@Scheduled(cron = '0 0 8 * * *')) comparing system date against registered deadlines.
- **Urgency Categorization:** Classifies alerts into Critical (≤ 3 days), Warning (≤ 7 days), and Upcoming (≤ 15 days), populating real-time notification bells.
- **Unread Alert Synchronization:** Real-time mark-as-read state synced with PostgreSQL.

Feature 9: WhatsApp Weekly Action Digest & Family/Parent Mode

Components & Source: FamilyModeView.tsx | WeeklyDigestModal.tsx | WeeklyDigest.java

Bridges the information gap between modern digital portals and non-technical parents and family members.

Key Mechanics:

- **WhatsApp Optimized Formatting:** Automatically generates emoji-structured text summaries ready for one-click sharing on WhatsApp and email.
- **Dual-Persona Translation:** Seamlessly toggles between English and colloquial Hindi (■■■■■■■) so parents understand upcoming deadlines and financial savings.
- **Family Guidance Dashboard:** Presents simplified financial overviews without overwhelming jargon.

Feature 10: Adaptive AI Study Timetable & Scheduling Chatbot

Components & Source: StudyChatbot.tsx | ChatbotService.java | studyChatbotData.ts

A comprehensive study timetable and exam planning AI assistant accessible directly via a floating interface on all screens.

Key Mechanics:

- **Triple-Provider LLM Integration:** Supports Google Gemini 1.5 Flash / Pro, Groq Llama 3.3, and OpenRouter for real-time natural language query resolution.
- **Built-in Offline Mentor:** Features 30 pre-indexed academic timetables, branch schedules (B.Tech, Medical, Arts), and backlog recovery plans requiring zero external API keys.
- **One-Click Routine Export:** Enables students to copy formatted daily study routines directly to their clipboard or calendar.

Feature 11: Defensive Security Headers & Web SPA Architecture

Components & Source: SecurityHeadersFilter.java | WebSpaConfig.java

Production-grade defensive headers filter and client-side history routing resolver compliant with OWASP and automated security scanners (ArgosX).

Key Mechanics:

- **Baseline Defensive Headers:** Injects X-Frame-Options: DENY, Strict-Transport-Security (HSTS max-age=63072000), X-Content-Type-Options: nosniff, Permissions-Policy, and Referrer-Policy.
- **Comprehensive CSP:** Implements Content Security Policy restricting script, style, font, connect, and iframe embedding sources.
- **SPA Fallback:** Routes all non-API HTML5 history requests directly to index.html while strictly enforcing authentication on /api/** endpoints.

Feature 12: Secure User Authentication & Session Pipeline

Components & Source: AuthController.java | AuthService.java | AuthenticationInterceptor.java

Handles multi-user accounts, persistent profile storage, and cryptographic protection.

Key Mechanics:

- **BCrypt Password Hashing:** Securely salts and hashes user credentials before storage in PostgreSQL users table.
- **Interceptor-Enforced Endpoints:** Protects sensitive endpoints (/api/profile, /api/opportunities, /api/digest, /api/deadline-alerts) requiring valid authentication tokens or returning HTTP 401.
- **Session State Cleanup:** Full client-side session erasure on logout resetting state to guest mode.

3. REST API & Database Schema Summary

The following table summarizes the primary backend endpoints and their operational contracts:

Endpoint Route	Method	Auth Required	Purpose & Payload
`/api/auth/register`	POST	Public	Registers new user with BCrypt hashed credentials
`/api/auth/login`	POST	Public	Validates credentials and issues session token
`/api/profile`	GET / PUT	Strict (Bearer/Session)	Fetches or updates detailed student academic profile
`/api/opportunities`	GET	Strict	Returns verified database opportunities with quotas
`/api/digest`	GET	Strict	Generates weekly action digest and WhatsApp summary
`/api/deadline-alerts`	GET	Strict	Fetches prioritized deadline notifications and counts
`/api/chatbot/query`	POST	Strict	Executes Gemini/Groq study schedule generation
`/health`	GET	Public	Container health check for Render uptime monitoring

4. Production Deployment & Operational Characteristics

GyanSetu is deployed on Render PaaS using an optimized 2-stage Docker container. The runtime leverages eclipse-temurin:17-jre-alpine with tuned memory parameters (-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0) that operate reliably under Render's free tier 512MB RAM constraints. The application maintains high accessibility (WCAG 2 AA compliant), sub-second page responsiveness, and complete data privacy protection.